VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY



FACULTY OF INFORMATION TECHNOLOGY

LAB REPORT

Lab 01: Xv6 User Programs

Instructor:
Leader: Phuc-Khang Nguyen 24120068
Members: Minh-Trong Pham 24120151
Trong-Nghia Hoang 24120103

Ho Chi Minh City, Ngày 15 tháng 10 năm 2026

Table of Contents

1 primes.c **1**

1.1 Tổng quan 1

1.2 Khó khăn và Thử thách 2

2 tree.c **3**

2.1 Tổng quan 3

2.2 Khó khăn và Thử thách 4

3 diff.c **7**

3.1 Tổng quan 7

3.2 Khó khăn và Thử thách 8

4 du.c **9**

4.1 Tổng quan 9

4.2 Khó khăn và Thử thách 10

5 PingPong (pingpong.c) **12**

5.1 Cách thực hiện 12

5.2 Xử lý lỗi và tài nguyên 14

5.3 Kiểm thử và kết quả mong đợi 14

6 Copy file cơ bản - cp (cp.c) **15**

6.1 Cách thực hiện 15

6.2 Xử lý lỗi và giới hạn 16

6.3 Kiểm thử và kết quả mong đợi 17

1 primes.c

1.1 Tổng quan

`primes.c` được triển khai in ra các số nguyên tốt từ `2...280` bằng thuật toán Sàng Eratosthenes đồng thời (*Concurrent Prime Sieve*) dựa trên mô hình xử lý luồng dữ liệu qua chuỗi đường ống dẫn (*concurrent pipeline*). Thuật toán hoạt động dựa trên ý tưởng cốt lõi là kết hợp đường ống Unix (*pipe*) với thuật toán sàng số nguyên tố, được đề xuất bởi Doug McIlroy, một nhà khoa học máy tính tại Bell Labs và là người phát minh ra khái niệm Unix Pipeline [1, 2]. Mô hình này đại diện cho kiến trúc tính toán dòng dữ liệu CSP (*Communicating Sequential Processes*) được khởi xướng bởi C. A. R. Hoare [3], đồng thời vận dụng trực tiếp các cơ chế quản lý tiến trình và giao tiếp liên tiến trình (IPC) trong hệ điều hành xv6 [4, 5].

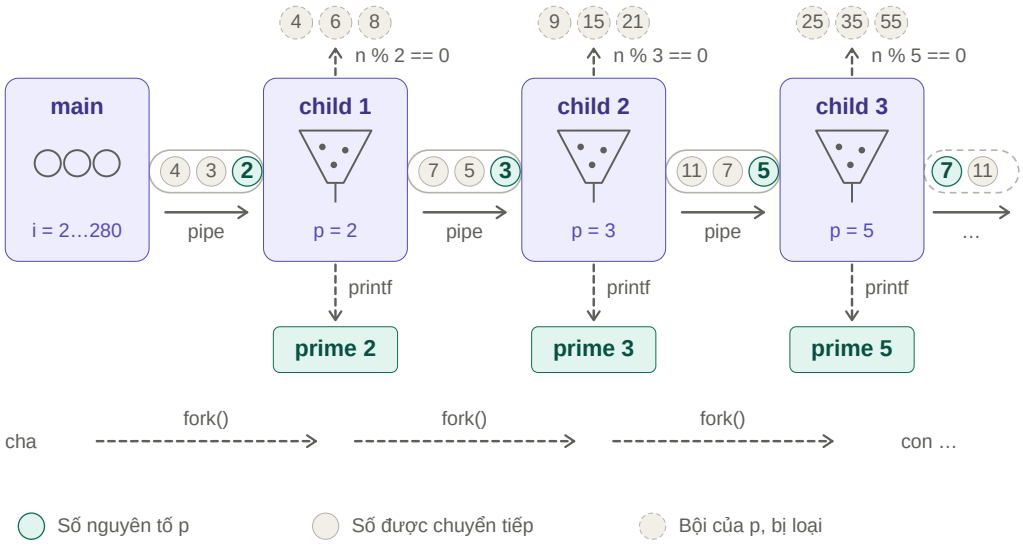


Figure 1: Pipeline sàng nguyên tố đồng thời trong `primes.c`

Luồng thực thi diễn ra tuần tự qua các giai đoạn:

- **Khởi tạo:** Tạo `pipe 0`, thực hiện `fork()` để tạo tiến trình con lọc đầu tiên, sau đó ghi dãy số từ 2 đến 280 vào đầu ghi của `pipe 0`.
- **Bước lọc:** Mỗi tiến trình đọc số nguyên đầu tiên p từ pipe bên trái (`left_fd`). Theo giả thuyết, ban đầu số 2 là số đầu tiên, và cũng là số nguyên tố, do đó sau khi lọc qua các bội

của 2 thì số p tiếp theo chắc chắn là số nguyên tố và được in ra màn hình. Kế tiếp, tiến trình tạo một pipe mới (`new_pipe`), `fork()` tiến trình con kế tiếp, rồi tiếp tục quá trình lọc: đọc các số n còn lại, nếu $n \pmod p \neq 0$ thì đẩy vào `new_pipe`.

- **Kết thúc:** Khi đường ống bên trái đóng hoàn toàn, hàm `read()` trả về giá trị ≤ 0 (EOF). Tiến trình thực hiện đóng đầu pipe, gọi `wait(0)` để đợi tiến trình con và thoát an toàn.

1.2 Khó khăn và Thử thách

Dựa trên các đặc thù kiến trúc và giới hạn tài nguyên của hệ điều hành xv6 [4], trong quá trình làm và triển khai thì nhóm có thấy được một vài điểm cần lưu ý:

1. **Vấn đề Deadlock:** Khi `write(pipe[1], ...)` mà chưa có `read(pipe[0], ...)` đọc dữ liệu, hoặc khi một tiến trình con chưa đóng hoàn toàn đầu ghi pipe của nó, hệ thống có thể rơi vào trạng thái *deadlock*, tức là không tiến trình nào có thể tiếp tục thực thi.
   Giải pháp: Khởi tạo `fork()` trước khi bắt đầu ghi, giúp luồng dữ liệu được tiêu thụ đồng thời dạng streaming.

2. **Quản lý File Descriptor và EOF:** Hàm `read()` chỉ trả về 0 khi toàn bộ các đầu ghi của pipe đó được đóng. Nếu tiến trình con không đóng đầu ghi thừa kế thừa từ cha, lệnh `read()` sẽ chờ vĩnh viễn. Ngoài ra, xv6 giới hạn mỗi process chỉ có 16 FDs (`NOFILE`).
   Giải pháp: Tiến trình con lập tức `close(new_pipe[1])` và `close(left_fd)`; tiến trình cha `close(new_pipe[0])` và `close(new_pipe[1])` ngay sau khi đẩy hết dãy số.

3. **Hiện tượng Zombie Process và Cơ chế kết thúc dây chuyền (Cascading Termination):**
   Trong thuật toán này, mỗi số nguyên tố tìm thấy sẽ sinh ra một tiến trình mới, tạo thành một chuỗi ống dẫn sâu gồm hàng chục tiến trình lồng nhau (tiệm cận giới hạn `NPROC = 64` của xv6 [4]). Nếu tiến trình cha kết thúc trước khi tiến trình con hoàn tất hoặc không thu hồi con, bảng tiến trình của hệ điều hành sẽ bị cạn kiệt do ngập tràn các tiến trình ma (*Zombie process*), đồng thời shell xv6 sẽ in dấu nhắc lệnh xen ngang kết quả in của các tiến trình con. Ngược lại, nếu cha gọi `wait()` trước khi đóng hoàn toàn đầu ghi, tiến trình con sẽ chờ `read()` vô tận gây ra lỗi treo hệ thống (*deadlock*).
   Giải pháp: Thiết lập thứ tự thu hồi tài nguyên dạng dây chuyền từ dưới lên (*bottom-up*): tiến trình cha chỉ gọi `wait(0)` sau khi đã ghi hết số và đóng đầu ghi (báo hiệu EOF xuôi dòng). Tiến trình con nhận EOF, chuyển tiếp cho tầng sau rồi mới đợi con của nó và thoát,

bảo đảm mọi tiến trình đều được giải phóng tuần tự theo thứ tự ngược lại trước khi `main` trả quyền điều khiển về cho hệ điều hành.

2 tree.c

2.1 Tổng quan

Lệnh `tree` dùng để hiển thị cấu trúc thư mục dạng cây, hỗ trợ hiển thị cấu trúc phân nhánh trực quan cùng các cờ tùy chọn `-L <depth>` (giới hạn độ sâu) và `-d` (chỉ hiển thị thư mục).

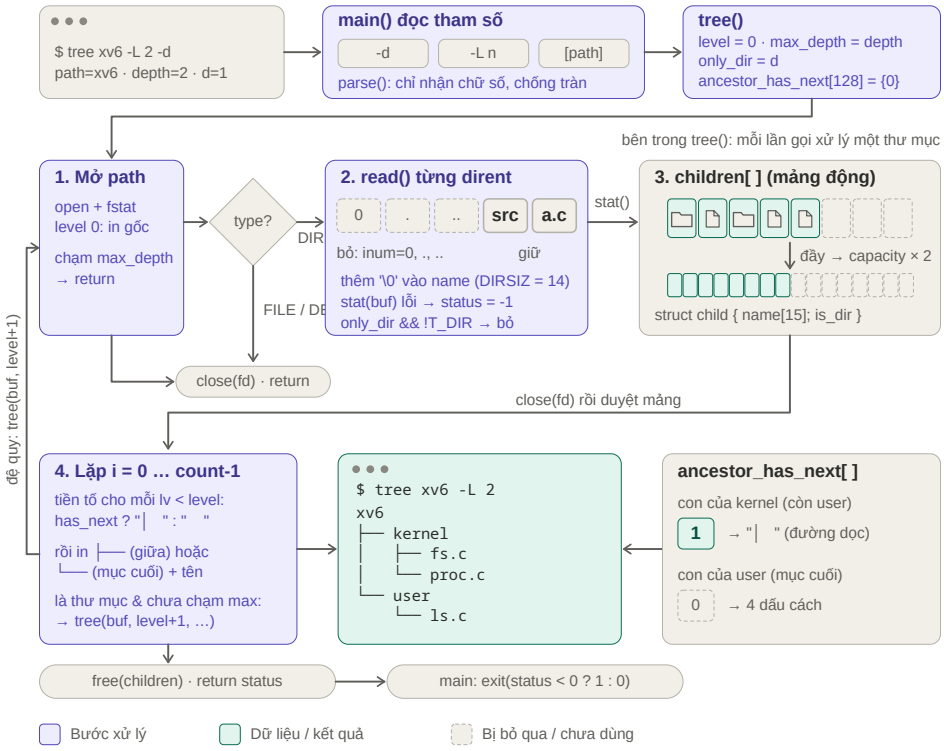


Figure 2: Sơ đồ luồng thực thi (Workflow) duyệt đệ quy DFS và định dạng nhánh cây trong `tree.c`

Luồng thực thi diễn ra tuần tự qua các giai đoạn:

- **Phân tích đối số (Argument Parsing):** Tiếp nhận các đối số dòng lệnh, xác định đường dẫn mục tiêu (mặc định là `"."`), các cờ `-d` và `-L <depth>`. Hàm `parse()` kiểm tra tính hợp lệ của chuỗi số và chủ động phát hiện tràn số nguyên `INT_MAX`.

- **Thu thập và Lọc danh mục con:** Mở thư mục với `O_RDONLY`, đọc tuần tự từng bản ghi `struct dirent`, lọc bỏ liên kết vòng lặp (`"."`, `".."`) và inode rác (`inum == 0`). Danh sách các mục con hợp lệ được lưu trữ an toàn trong mảng động tự co giãn `children` trên heap.
- **Định dạng phân nhánh và Đệ quy sâu (Hierarchical DFS Traversal):** Thực hiện `close(fd)` ngay khi hoàn tất đọc dữ liệu để giải phóng tài nguyên hệ thống. Duyệt mảng `children`, kết xuất các ký tự định dạng nhánh cây chuẩn (`|-`, `\___`, `|  `) dựa trên mảng trạng thái `ancestor_has_next`, sau đó tiến hành gọi đệ quy `tree()` cho các thư mục con theo chiều sâu.

2.2 Khó khăn và Thử thách

Dựa trên kiến trúc của hệ điều hành xv6 và các trường hợp phức tạp trong quản lý tệp tin [4], nhóm nhận thấy các khó khăn như sau:

1. **Giới hạn File Descriptor (`NOFILE = 16`):**
   Trong kernel xv6, mỗi tiến trình bị giới hạn cứng chỉ được mở tối đa 16 file descriptors đồng thời (`NOFILE = 16`) [4]. Nếu duy trì `fd` của thư mục đang mở trong khi gọi đệ quy sâu xuống các thư mục con, hệ thống sẽ nhanh chóng cạn kiệt FD khi độ sâu cây vượt quá ngưỡng cho phép, gây lỗi sập chương trình.
   Giải pháp: Đọc toàn bộ các mục con vào mảng động `children`, sau đó lập tức gọi lệnh `close(fd)` trước khi duyệt in và thực hiện bất kỳ lời gọi đệ quy nào. Kỹ thuật này giúp số lượng FD mở tại mỗi thời điểm chỉ là $O(1)$ (tối đa 1 FD trong lúc đọc), bảo đảm an toàn tuyệt đối với mọi độ sâu cấu trúc thư mục.

2. **Xử lý chuỗi tên tệp thiếu ký tự kết thúc (`DIRSIZ = 14` Null-termination):**
   Cấu trúc `struct dirent` của xv6 quy định mảng tên tệp `name[DIRSIZ]` với `DIRSIZ = 14`. Khi một tệp có tên đạt đúng 14 ký tự, hệ thống tệp không lưu ký tự kết thúc chuỗi `'\0'`. Nếu thao tác trực tiếp bằng các hàm chuỗi tiêu chuẩn như `strlen()`, `strcpy()` hay `printf("%s")`, chương trình sẽ đọc tràn sang vùng nhớ lân cận (*buffer over-read*), dẫn đến hiển thị ký tự rác hoặc sai lệch đường dẫn.
   Giải pháp: Khởi tạo bộ đệm trung gian có kích thước `DIRSIZ + 1`, sao chép chính xác 14 bytes bằng `memmove(name, de.name, DIRSIZ)` và chủ động kết thúc chuỗi an toàn bằng lệnh `name[DIRSIZ] = 0` trước khi ghép chuỗi hay kiểm tra `stat()`.

3. **Kiểm soát tràn bộ đệm đường dẫn (`PATHSIZ = 128`) và Tính chịu lỗi (Fault Tolerance):**
   Hệ thống giới hạn mảng đệm đường dẫn `PATHSIZ = 128`. Khi cây thư mục có nhiều cấp lồng nhau, việc nối thêm tên con có thể vượt quá kích thước bộ đệm. Cách xử lý thoát đột ngột (`exit`) sẽ làm gián đoạn toàn bộ quá trình duyệt cây của các nhánh khác.
   Giải pháp: Chương trình chủ động tính toán trước: nếu chiều dài đường dẫn cha cộng tên con và các ký tự phân tách vượt quá `sizeof(buf)`, hệ thống chỉ ghi cảnh báo ra `stderr`, đánh dấu mã trạng thái lỗi (`status = -1`), bỏ qua entry con bị tràn (`continue`) và tiếp tục duyệt các tệp tin hợp lệ còn lại trong cùng thư mục.

4. **Quản lý bộ nhớ và Mảng động:**
   Số lượng tập tin trong một thư mục là biến thiên và không thể xác định trước. Việc sử dụng mảng tĩnh kích thước lớn trên stack dễ gây tràn stack, trong khi mảng cố định quá nhỏ lại không thể chứa hết các thư mục lớn.
   Giải pháp: Sử dụng mảng động `children` với dung lượng khởi điểm capacity $= 8$. Khi số lượng phần tử đạt ngưỡng (`count == capacity`), chương trình nhân đôi dung lượng (capacity $\times 2$), cấp phát vùng nhớ mới, sao chép dữ liệu cũ qua `memmove()` và thu hồi vùng nhớ cũ bằng `free()`. Đồng thời, chương trình luôn kiểm tra con trỏ trả về từ `malloc()` để kịp thời báo lỗi nếu cạn kiệt heap, và giải phóng mảng ngay sau khi hoàn tất cấp duyệt.

5. **Định dạng phân nhánh và truy vết các thư mục cha:**
   Để vẽ đúng định dạng cây phân cấp (phần tử giữa dùng `|-`, phần tử cuối dùng `\___`), mỗi node cần xác định chính xác ở từng cấp tổ tiên phía trước còn phần tử tiếp theo hay không, và bản thân node có phải là phần tử cuối cùng của thư mục hiện tại hay không.
   Giải pháp: Sử dụng mảng cờ `char ancestor_has_next[PATHSIZ]`. Trước khi gọi đệ quy cấp level $+ 1$, chương trình lưu trạng thái `ancestor_has_next[level] = !is_last`. Khi in từng dòng, chương trình duyệt qua các cấp tổ tiên: in vạch dọc `|  ` nếu cấp đó còn phần tử phía sau, ngược lại in khoảng trống thụt lề. Nút hiện tại được kết xuất bằng `\___ ` nếu là phần tử cuối (`i == count - 1`) hoặc `|- ` nếu còn phần tử kế tiếp.

6. **Phân tích đối số và tràn số (*Integer Overflow*):**
   Giá trị độ sâu nhập vào từ cờ `-L <depth>` có thể chứa ký tự không hợp lệ, chuỗi rỗng hoặc số quá lớn vượt quá giới hạn biểu diễn của số nguyên 32-bit có dấu.
   Giải pháp: Hiện thực hàm `parse(char *s)` an toàn: kiểm tra chuỗi rỗng `*s == '\0'`, xác thực từng ký tự thuộc khoảng `'0'` đến `'9'`, và chủ động phát hiện tràn số trước mỗi

phép nhân 10: `depth > (INT_MAX - digit) / 10`. Đồng thời bắt lỗi cú pháp dòng lệnh (thiếu tham số cho `-L`, lặp đường dẫn), in thông báo `usage` và thoát với mã lỗi.

7. **Xử lý tệp tin hỏng và Đọc đĩa dở dang:**
   Trong quá trình quét thư mục, một tệp tin con có thể gặp lỗi phân quyền hoặc tệp bị hỏng khiến `stat()` thất bại, hoặc thao tác đọc trả về số byte dở dang không đủ kích thước của một `struct dirent`.
   Giải pháp: Khi `stat()` thất bại, chương trình ghi nhận lỗi `cannot stat`, cập nhật mã trạng thái `status = -1` và tiếp tục duyệt các tệp khác thay vì dừng toàn bộ chương trình. Đồng thời kiểm tra biến `bytes_read != 0` sau vòng lặp đọc để phát hiện lỗi đọc đĩa và trả mã thoát chuẩn `exit(status < 0 ? 1 : 0)`.

3 diff.c

3.1 Tổng quan

`diff.c` được hiện thực nhằm so sánh hai tập tin văn bản theo từng dòng chuẩn POSIX [6], dựa trên nguyên lý so khớp chuỗi ký tự trong các hệ thống Unix [7]. Mục tiêu chính là phát hiện vị trí chênh lệch, in chi tiết nội dung phân kỳ hoặc ngắt nhanh để kiểm tra tính toàn vẹn của tệp tin.

Trong hệ điều hành xv6, thư viện chuẩn không hỗ trợ các hàm đọc dòng như `getline()` hay `fgets()` [4]. Do đó, nhóm đã tự xây dựng hàm `readline()` đọc trực tiếp từng byte từ file descriptor kết hợp với kỹ thuật cấp phát bộ đệm tự co giãn trên heap.

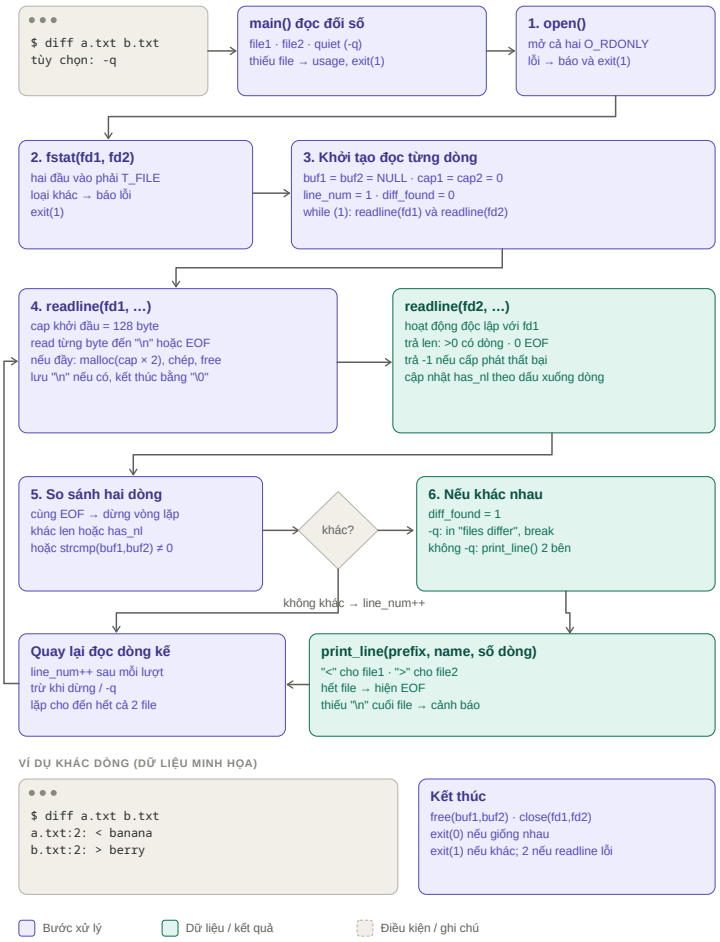


Figure 3: Sơ đồ luồng thực thi (Workflow) và cơ chế so khớp dòng trong `diff.c`

Luồng thực thi diễn ra tuần tự qua các giai đoạn:

- **Khởi tạo và kiểm tra tệp:** Phân tích đối số dòng lệnh, mở `fd1`, `fd2` với cờ `O_RDONLY` và dùng `fstat()` kiểm tra cả hai đều là tệp thường (`st.type == T_FILE`), tránh đọc nhầm vào thư mục hay thiết bị.
- **Đọc dòng song song và cấp phát động:** Khởi tạo hai bộ đệm độc lập 128 bytes. Khi dòng vượt sức chứa (`len + 1 >= cap`), bộ đệm tự nhân đôi dung lượng qua `malloc()` và `memmove()`, giải phóng vùng nhớ cũ an toàn.
- **So khớp và in kết quả:** Tại mỗi bước lặp, so sánh chiều dài, cờ `has_newline` và gọi `strcmp()`. Nếu khác biệt, hàm `print_line()` in tiền tố `<` cho tệp 1 và `>` cho tệp 2 kèm chỉ số dòng hiện hành.

3.2 Khó khăn và Thử thách

Dựa trên kiến trúc của hệ điều hành xv6 và các trường hợp phức tạp khi xử lý tệp văn bản [4], nhóm nhận thấy các khó khăn như sau:

1. **Xử lý lệch số dòng giữa hai tệp:** Khi hai tệp lệch số dòng, một bên sẽ gặp EOF (`len == 0`) trước. Cách so sánh thông thường dễ đọc dữ liệu rác hoặc dừng sớm làm sót nội dung chênh lệch ở tệp dài hơn.
   Giải pháp: Hàm `print_line()` hỗ trợ nhận con trỏ rỗng (`line == 0`). Khi một tệp hết trước, chương trình in chuỗi chuẩn `EOF` (ví dụ `f1:4: < EOF`), đồng thời duy trì vòng lặp đọc và in dữ liệu thực tế của tệp còn lại cho đến khi cả hai kết thúc hoàn toàn.
2. **Xử lý dòng cuối thiếu ký tự ngắt dòng:** Tệp tin thiếu ký tự `'\n'` ở dòng cuối có thể làm ghép sai nội dung theo chuẩn POSIX.
   Giải pháp: Hàm `readline()` quản lý cờ trạng thái `has_newline`. Nếu dòng cuối không chứa `'\n'`, chương trình chủ động ngắt dòng và in cảnh báo: `diff: <file>:<line>: no newline at end of file`.
3. **Tối ưu cờ ngắt nhanh (`-q`) và mã thoát:** Trong kiểm thử tự động, chương trình chỉ cần kết luận hai tệp có tương đương hay không mà không cần tiêu tốn I/O để in chi tiết.
   Giải pháp: Khi bật cờ `-q`, chương trình ngắt vòng lặp ngay tại chênh lệch đầu tiên (`is_diff = 1`) và in `diff: files differ`. Toàn bộ bộ đệm động và FD đều được giải phóng trước khi thoát. Chương trình trả về mã thoát chuẩn: `exit(0)` (trùng khớp), `exit(1)` (khác biệt) và `exit(2)` (lỗi I/O).

4 du.c

4.1 Tổng quan

`du.c` (*Disk Usage*) mô phỏng tiện ích cốt lõi trên các hệ điều hành họ Unix theo chuẩn POSIX [8], hỗ trợ ước tính dung lượng chiếm dụng của cây thư mục và tập tin bằng thuật toán duyệt đệ quy hậu thứ tự (*post-order bottom-up traversal*). Tổng dung lượng của một thư mục được định nghĩa bằng tổng dung lượng thực tế của tất cả các tệp tin và cây con bên trong nó.

Trong hệ thống tệp xv6 [4], một thư mục được tổ chức như một tập tin đặc biệt chứa danh sách các bản ghi `struct dirent` (16 bytes gồm 2 bytes chỉ mục `inum` và 14 bytes tên `name`). Kích thước byte thực tế `st.size` của từng inode được truy xuất chính xác thông qua lời gọi hệ thống `stat()`.

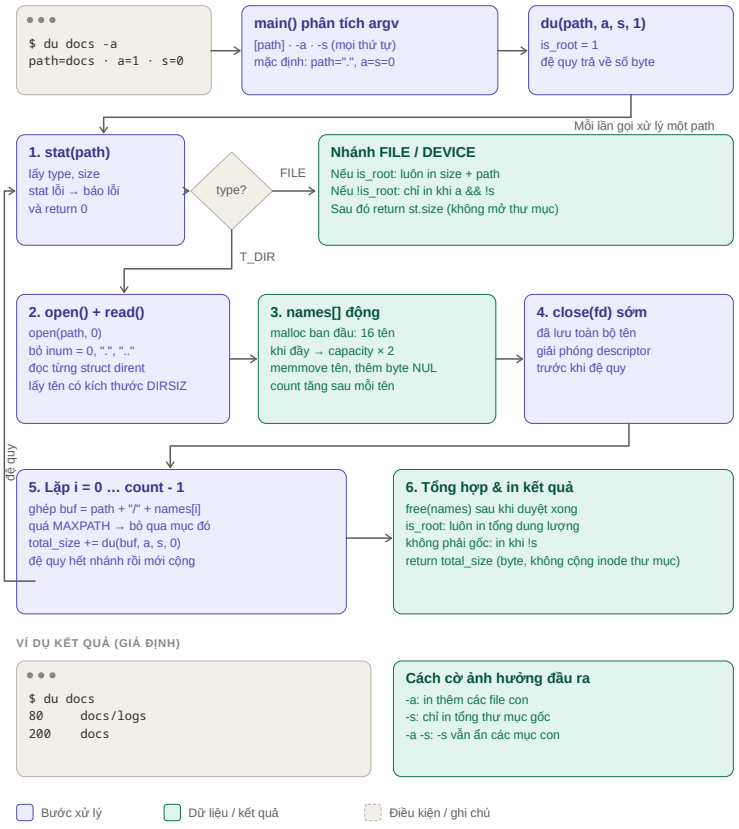


Figure 4: Sơ đồ luồng thực thi (Workflow) duyệt đệ quy Bottom-up và giải phóng FD sớm trong `du.c`

Luồng thực thi diễn ra tuần tự qua các giai đoạn:

- **Khởi tạo và phân tích đối số:** Tiếp nhận đường dẫn mục tiêu (mặc định là `"."`), phân tích cờ tóm tắt `-s` (chỉ in kích thước thư mục gốc) và cờ chi tiết `-a` (in toàn bộ kích thước tệp tin đơn lẻ lẫn thư mục con).
- **Điều kiện dừng (Base Case):** Sử dụng `stat(path, &st)`. Nếu đường dẫn trỏ tới một tệp tin thường (`T_FILE`) hoặc tệp thiết bị (`T_DEVICE`), kích thước `st.size` được trả về ngay lập tức; đồng thời in thông tin ra màn hình nếu đường dẫn là gốc được chỉ định hoặc có cờ `-a` (`is_root || (a && !s)`).
- **Duyệt con và cộng dồn (Bottom-up):** Khi gặp thư mục (`T_DIR`), chương trình quét nội dung thư mục, lưu danh sách các tên tệp con hợp lệ, tiến hành gọi đệ quy `du(buf, a, s, 0)` cho từng nhánh con và cộng dồn vào biến `total_size`. Thư mục hoàn thành đệ quy sẽ in kích thước tích lũy của chính nó trước khi trả kết quả về cho thư mục cha.

4.2 Khó khăn và Thử thách

Dựa trên kiến trúc của hệ điều hành xv6 và các ca biên trong quản lý tệp tin [4], nhóm nhận thấy các khó khăn như sau:

1. **Giới hạn File Descriptor (`NOFILE = 16`):** Trong kernel xv6, mỗi tiến trình bị giới hạn mở tối đa 16 file descriptors đồng thời (`NOFILE = 16` trong `kernel/param.h`) [4]. Nếu duy trì `fd` của thư mục đang mở trong suốt các lời gọi đệ quy sâu, hệ thống sẽ nhanh chóng cạn kiệt FD khi độ sâu thư mục vượt quá ngưỡng cho phép, gây lỗi sập chương trình.
   Giải pháp: Đọc toàn bộ tên các mục con vào mảng động `names` trên bộ nhớ heap, sau đó thực hiện lệnh `close(fd)` ngay lập tức trước khi bắt đầu bất kỳ lời gọi đệ quy con nào. Kỹ thuật này giúp số lượng FD mở tại mỗi thời điểm chỉ là $O(1)$ (tối đa 1 FD trong lúc đọc), bảo đảm an toàn với mọi độ sâu cấu trúc thư mục.
2. **Phòng chống lặp vô hạn và lọc Inode rác:** Mọi thư mục Unix đều chứa hai thực thể liên kết cứng đặc thù là `"."` (thư mục hiện hành) và `".."` (thư mục cha). Ngoài ra, các entry đã xóa trong xv6 vẫn tồn tại trong bảng chỉ mục với `inum == 0`.
   Giải pháp: Chủ động loại bỏ các khối có `de.inum == 0` và kiểm tra lọc bỏ chuỗi `"."` cùng `".."`, tránh hoàn toàn nguy cơ lặp đệ quy vô hạn làm tràn stack kernel và đảm bảo tính chính xác của phép cộng dồn dung lượng.
3. **Quản lý bộ nhớ mảng động và kiểm soát đường dẫn:** Số lượng phần tử trong một

thư mục là biến thiên và không thể biết trước, trong khi việc ghép nối chuỗi đường dẫn đệ quy tiềm ẩn nguy cơ tràn mảng đệm.

Giải pháp: Khởi tạo mảng con trỏ `names` ban đầu với sức chứa 16 phần tử, tự động tăng gấp đôi kích thước (`capacity * 2`) qua `malloc()` và `memmove()` khi danh sách vượt ngưỡng. Đồng thời, trước khi ghép chuỗi đường dẫn con, chương trình luôn kiểm tra điều kiện an toàn: `strlen(path) + 1 + strlen(names[i]) < sizeof(buf)` và giải phóng vùng nhớ sau khi sử dụng.

5 PingPong (pingpong.c)

5.1 Cách thực hiện

Chương trình dùng hai pipe để trao đổi một byte giữa tiến trình cha và con. Mỗi pipe chỉ truyền dữ liệu theo một chiều: đầu `[0]` dùng để đọc và đầu `[1]` dùng để ghi [4].

Cha tạo `parent_to_child` và `child_to_parent`, sau đó gọi `fork()`. Cha gửi byte `'p'` qua pipe thứ nhất rồi chờ phản hồi qua pipe thứ hai. Con nhận byte, in `<pid>: received ping`, gửi lại byte vừa nhận và thoát. Cha nhận phản hồi, in `<pid>: received pong`, rồi gọi `wait(&status_child)` để thu hồi con.

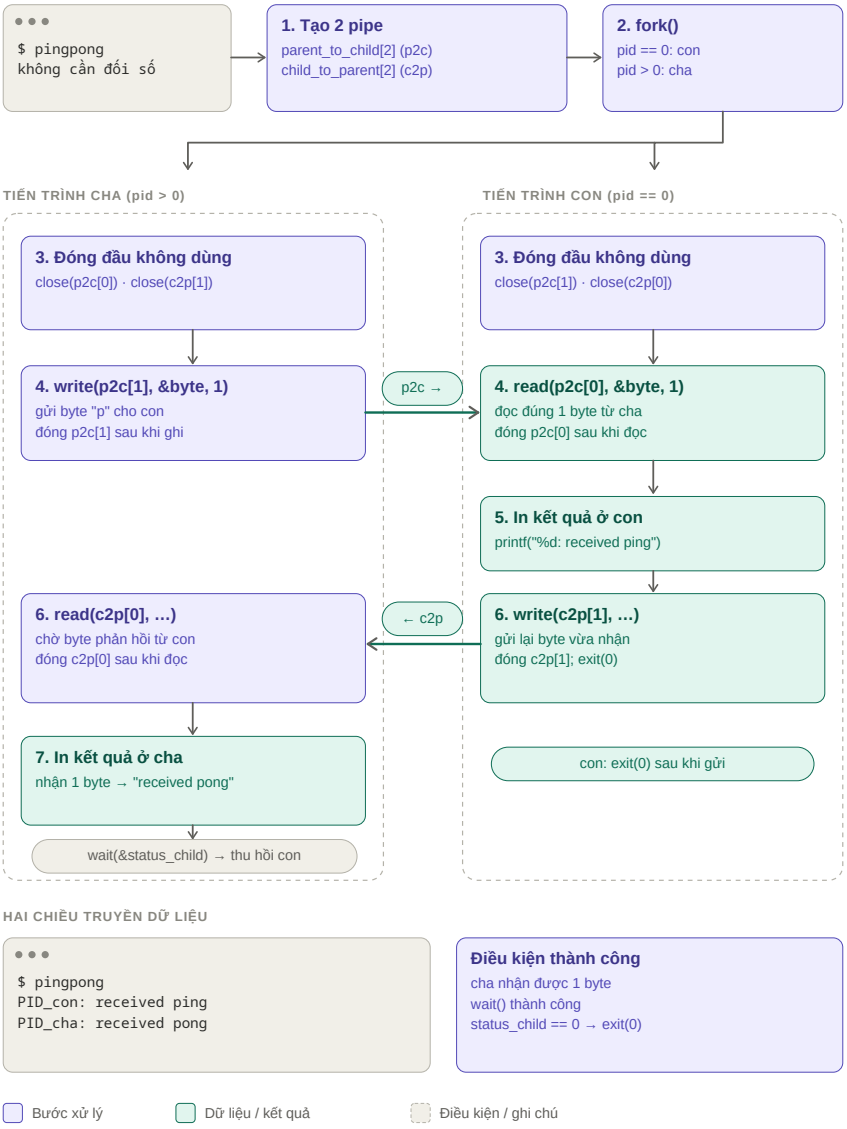


Figure 5: Sơ đồ luồng tương tác IPC qua hai pipe và thu hồi tiến trình con trong `pingpong.c`

Ngay sau `fork()`, mỗi tiến trình đóng các đầu pipe không sử dụng:

Pipe	Cha giữ	Con giữ
`parent_to_child`	Đầu ghi `[1]`	Đầu đọc `[0]`
`child_to_parent`	Đầu đọc `[0]`	Đầu ghi `[1]`

Table 1: Các đầu pipe được sử dụng sau khi tạo tiến trình con

Con in thông báo trước khi gửi phản hồi, còn cha chỉ in sau khi đọc được phản hồi. Vì vậy, trong luồng chạy thành công, dòng `received ping` xuất hiện trước dòng `received pong`, không phụ thuộc tiến trình nào được lập lịch trước.

5.2 Xử lý lỗi và tài nguyên

Chương trình kiểm tra kết quả tạo pipe và `fork()`. Nếu tạo pipe thứ hai thất bại, chương trình đóng hai đầu của pipe thứ nhất; nếu `fork()` thất bại, chương trình đóng cả bốn đầu pipe trước khi thoát với mã lỗi 1.

Các lần `read()` và `write()` đều yêu cầu đúng một byte. Giá trị trả về khác 1 được xem là trao đổi thất bại. Cha chỉ trả về mã thành công khi nhận đủ một byte và con kết thúc với trạng thái 0.

Việc đóng đầu ghi thừa giúp bên đọc nhận được EOF khi bên gửi đã kết thúc và dữ liệu trong pipe đã được đọc hết. Với yêu cầu đọc một byte ở bài này, nếu vẫn còn đầu ghi mở mà pipe không có dữ liệu, `read()` sẽ chờ. EOF được biểu hiện bằng giá trị trả về 0, không phải một tín hiệu ngắt.

Trong xv6, `exit()` tự đóng các file descriptor còn mở của tiến trình. Vì vậy, những nhánh lỗi thoát ngay mà chưa đóng tường minh từng FD không gây rò rỉ FD sau khi tiến trình kết thúc. Các lệnh `close()` trong chương trình giúp thể hiện rõ thời điểm không còn cần một đầu pipe.

Trong luồng thành công, `wait()` thu hồi tài nguyên của con trước khi cha thoát. Con có thể tạm thời ở trạng thái `ZOMBIE` từ lúc kết thúc đến khi được thu hồi. Riêng nhánh cha gửi byte thất bại, cha gọi `exit(1)` ngay, không gọi `wait()`; con được chuyển cho tiến trình `init` thu hồi.

5.3 Kiểm thử và kết quả mong đợi

Khi chạy `pingpong` trên shell xv6, kết quả mong đợi gồm hai dòng:

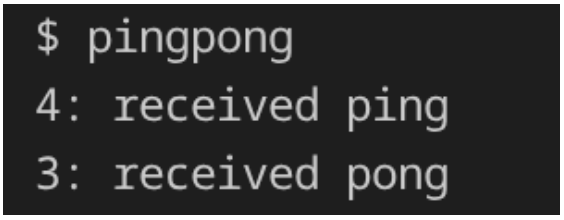


Figure 6: Kết quả chạy `pingpong` trên shell xv6

6 Copy file cơ bản - cp (cp.c)

6.1 Cách thực hiện

Lệnh có cú pháp `cp src dst`. Hàm `main()` yêu cầu đúng hai đối số ngoài tên chương trình; nếu không đúng, chương trình in `usage: cp src dst` và thoát với mã lỗi 1.

Hàm `cp()` mở nguồn bằng `O_RDONLY`, lấy thông tin nguồn bằng `fstat()` và từ chối nguồn là thư mục. Nếu lấy được thông tin đích bằng `stat()`, chương trình từ chối đích là thư mục và kiểm tra nguồn, đích có cùng cặp `(dev, ino)` hay không. Kiểm tra này được thực hiện trước khi mở đích để tránh xóa nội dung nguồn khi hai đường dẫn cùng trỏ tới một tệp

Sau các kiểm tra, chương trình mở đích với các cờ `O_WRONLY | O_CREATE | O_TRUNC`. Tệp đích được tạo nếu chưa tồn tại; nếu đã là tệp thường, nội dung cũ bị xóa để ghi nội dung mới [4].

Chương trình dùng buffer 512 byte theo gợi ý của đề. Đây là kích thước buffer ở user space; kích thước block của hệ thống tệp trong `kernel/fs.h` là `BSIZE = 1024`

Mỗi vòng lặp đọc tối đa 512 byte từ nguồn. Sau đó, vòng lặp ghi dùng `total_written` để theo dõi số byte đã ghi và tiếp tục ghi phần còn lại nếu cần. Chương trình chỉ đọc buffer tiếp theo sau khi đã ghi đủ số byte vừa đọc. Vì dùng số byte do `read()` trả về, lần đọc cuối không cần đủ 512 byte và tệp rỗng cũng được xử lý.

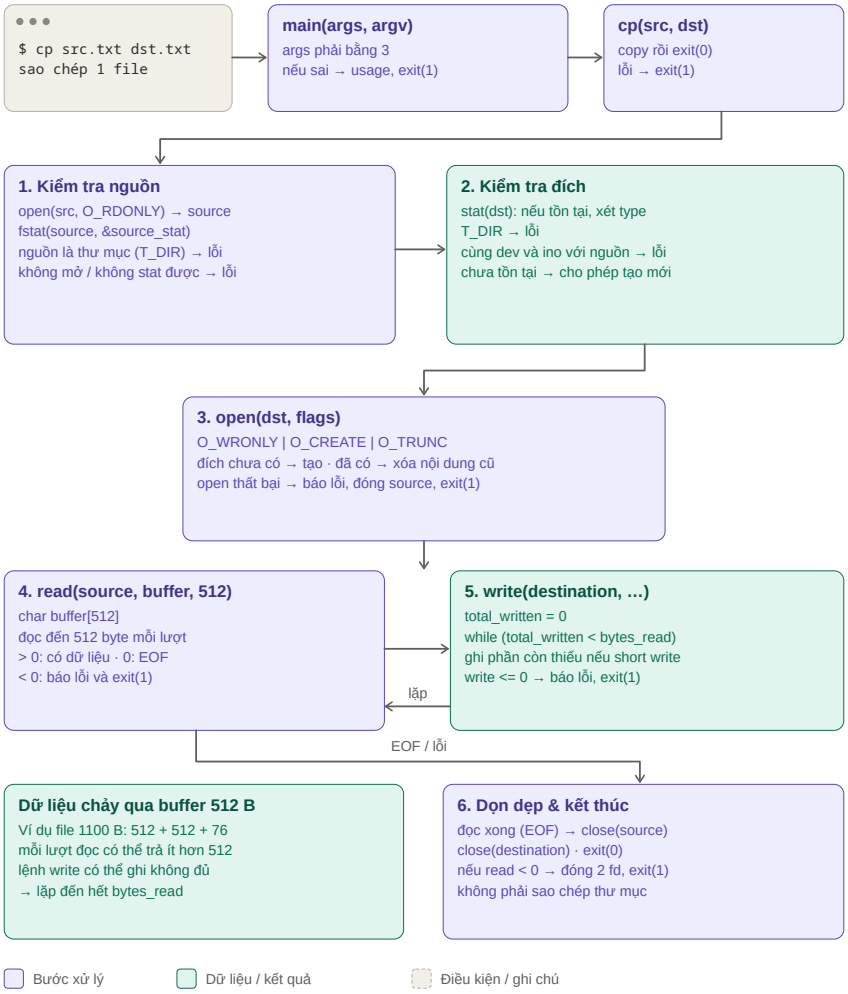


Figure 7: Sơ đồ luồng kiểm tra inode và sao chép dữ liệu theo bộ đệm trong `cp.c`

6.2 Xử lý lỗi và giới hạn

- Nếu mở nguồn, lấy thông tin nguồn hoặc mở đích thất bại, chương trình báo lỗi và thoát với mã 1. Khi mở đích thất bại, chỉ FD nguồn cần được đóng vì chưa có FD đích hợp lệ.
- Nếu đọc trả về số âm hoặc ghi trả về giá trị không dương, chương trình báo lỗi, đóng cả hai FD và thoát với mã 1. Đọc trả về 0 được xem là EOF bình thường; chương trình đóng hai FD và thoát với mã 0.
- Vòng lặp ghi có xử lý giá trị trả về dương nhỏ hơn số byte yêu cầu. Tuy nhiên, với tệp

thường trong phiên bản xv6 này, `filewrite()` trả về đủ số byte yêu cầu hoặc `-1`; khi báo lỗi, một phần dữ liệu vẫn có thể đã được ghi vào đích.

- Nếu lỗi I/O xảy ra sau khi mở đích, chương trình không khôi phục nội dung đích cũ và có thể để lại tệp sao chép dở. Báo lỗi và không khôi phục dữ liệu được
- Kiểm tra cùng tệp bằng `(dev, ino)` xử lý các đường dẫn ổn định. Chương trình chưa bảo vệ trường hợp tiến trình khác thay đổi đường dẫn đích giữa bước kiểm tra và bước mở đích.

6.3 Kiểm thử và kết quả mong đợi

Bảng dưới đây liệt kê các trường hợp cần kiểm tra trên xv6. Đây là kết quả mong đợi từ cách triển khai:

Kiểm thử	Kết quả mong đợi
`echo hello > a`, `cp a b`, `cat b`	In `hello` (sao chép thành công).
`cp a`	In `usage: cp src dst` (thiếu tham số).
`cp nofile out`	In `cp: cannot open nofile` (nguồn không tồn tại).
`cp . a`	In `cp: source is a directory: .` (nguồn là thư mục).
`cp a a`	In `cp: source and destination are the same file` (nguồn và đích trùng nhau).

Table 2: Các trường hợp cần kiểm tra chức năng của lệnh `cp`

```
$ echo hello > a
$ cp a b
$ cat b
hello
$
$ cp a
usage: cp src dst
$ cp nofile out
cp: cannot open nofile
$ cp . a
cp: source is a directory: .
$ cp a a
cp: source and destination are the same file
```

Figure 8: Kết quả chạy lệnh `cp` trên shell xv6

References

[1] M. D. McIlroy, "Coroutines: Semantics in terms of a hypothetical coroutine machine," *Bell Laboratories Memorandum*, 1968.

[2] M. D. McIlroy, E. N. Pinson, and B. A. Tague, "Unix time-sharing system: Foreword," *The Bell System Technical Journal*, vol. 57, no. 6, pp. 1899–1904, 1978.

[3] C. A. R. Hoare, "Communicating sequential processes," *Communications of the ACM*, vol. 21, no. 8, pp. 666–677, 1978.

[4] R. Cox, F. Kaashoek, and R. Morris, *xv6: a simple, Unix-like teaching operating system*, 2024. [Online]. Available: https://pdos.csail.mit.edu/6.828/2024/xv6/book-riscv-rev4.pdf

[5] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books, 2018.

[6] IEEE and The Open Group, *diff - compare two files*, 2018, the Open Group Base Specifications Issue 7, 2018 edition / IEEE Std 1003.1-2017.

[7] J. W. Hunt and M. D. McIlroy, "An algorithm for differential file comparison," Bell Laboratories, Tech. Rep. 41, 1976.

[8] IEEE and The Open Group, *du - estimate file space usage*, 2018, the Open Group Base Specifications Issue 7, 2018 edition / IEEE Std 1003.1-2017.